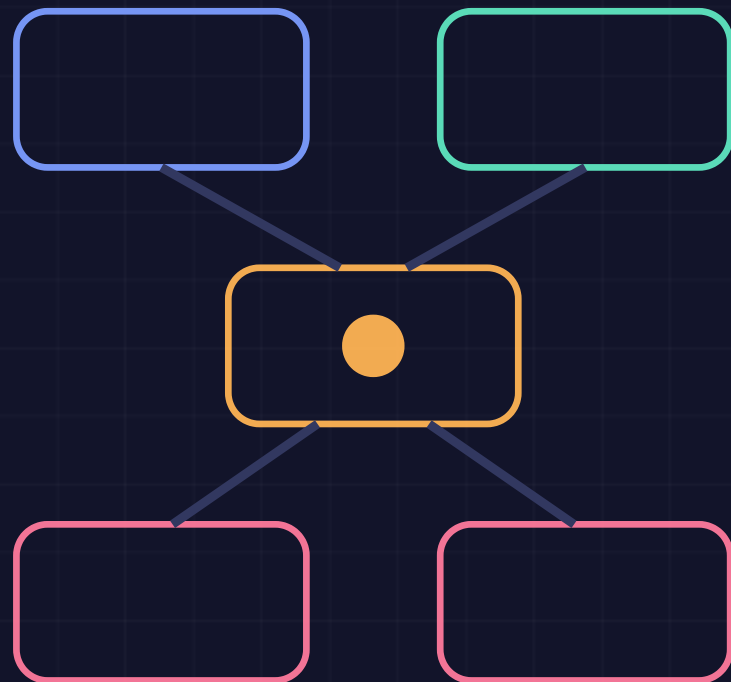


Neuron. How it is built

A Python pipeline turns hand-written lesson files into checked data. A React app reads that data through a small set of domain services, and two builds ship it: an installable web app and a single offline HTML file.



Neuron · repository [miekuskamil/ai-academy-game](#)

Build reviewed: branch [review-fixes](#), commit [474e92a](#)

26 September 2026 · document 2 of 3 ([Overview](#) · [Architecture](#) · [Logic](#))

From lesson files to a phone

Content and code are kept apart. Lessons are plain YAML written by a person; a Python build checks them and compiles one JSON file; the app never parses YAML and never trusts unchecked content.

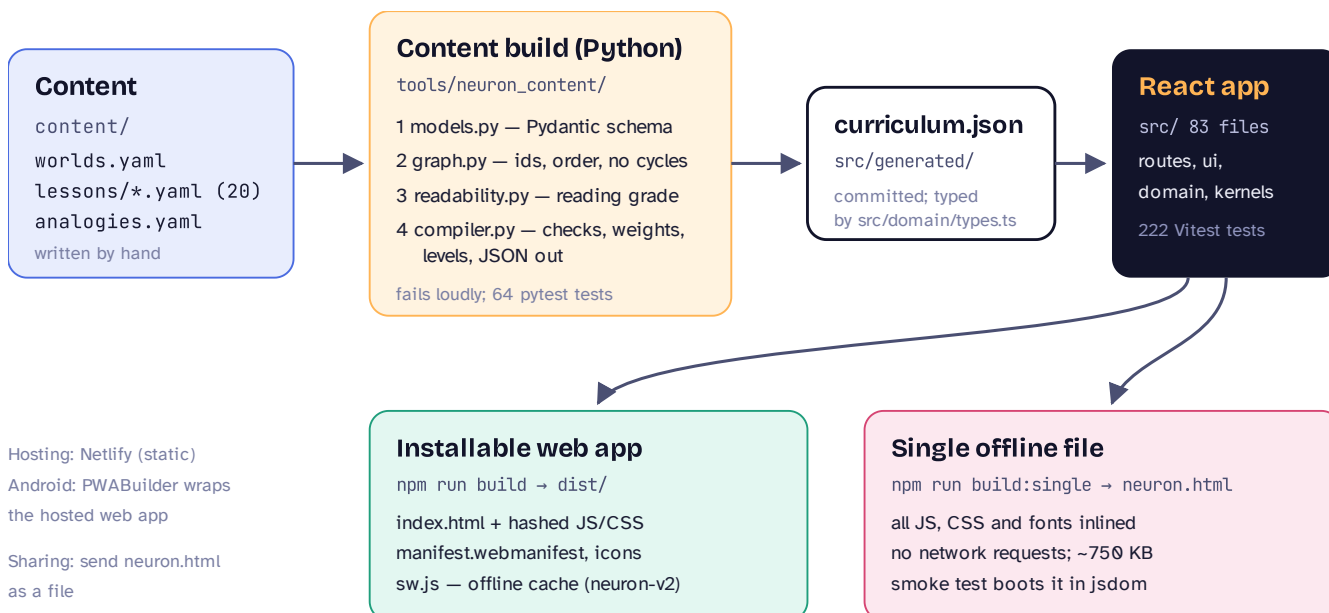


Figure 1. The build path. Solid arrows are build steps; nothing on this path runs in the child's browser except the app itself.

Technology

AREA	CHOICE	WHY
Content build	Python 3, Pydantic, PyYAML, pytest	Strict schema and clear errors for hand-written lessons.
App	React 18, TypeScript (strict), Vite	Fast static bundle; types generated from the same schema.
Styling	Tailwind 3 mapped to CSS design tokens	One token change restyles everything; no stock template look.
Routing	Small hash router in <code>src/lib/router.tsx</code>	Works from <code>file://</code> , subpaths and WebViews with no server.
Fonts	Bricolage Grotesque, Atkinson Hyperlegible, JetBrains Mono (bundled)	Readable for young readers; no font requests at runtime.
Tests	Vitest + Testing Library; pytest; Playwright walk	Logic, screens and the real built file are all exercised.

Who may talk to whom

The app is split into four layers. Dependencies point one way only: screens use services; services never import screens. A lint rule enforces this, so domain logic stays testable without a browser.

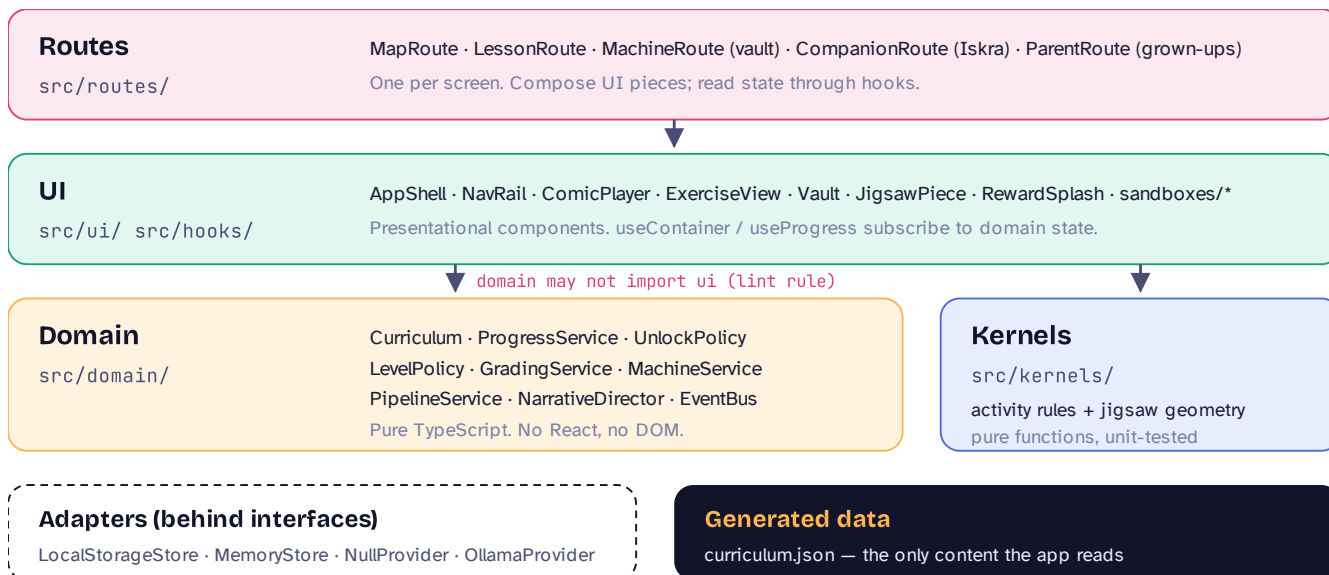


Figure 2. Layers. Each layer may use the ones below it. The domain layer is checked by ESLint's `no-restricted-imports` so it can never import UI or React.

Folder map

PATH	CONTENTS
content/	World list, 20 lesson files, shared analogies. The only place lesson text lives.
tools/neuron_content/	Python content build: schema, graph checks, reading-level gate, compiler.
src/domain/	Services, policies, grading, progress storage and migrations, event bus, composition root.
src/kernels/	Rules for each activity (prompt lab, made-up answers, sorter, checker, dial, pipeline, agent) and the jigsaw geometry.
src/ui/	Shell, navigation, comic player, question views, vault, reward pop-up, activities, settings panels.
src/routes/	The five screens: map, lesson, puzzle (vault), Iskra, grown-ups.
src/styles/	Design tokens (colours, spacing, radii, type, motion) and global CSS with motion guards.
public/	PWA manifest, service worker, icons, favicon.
scripts/	Single-file inliner, size gate, style-class check, smoke test.

One place wires everything

`createContainer()` in `src/domain/container.ts` is the only place concrete classes are named. Screens ask the container for behaviour through interfaces, which is what lets every test run against in-memory fakes and lets storage be swapped for a native store later.

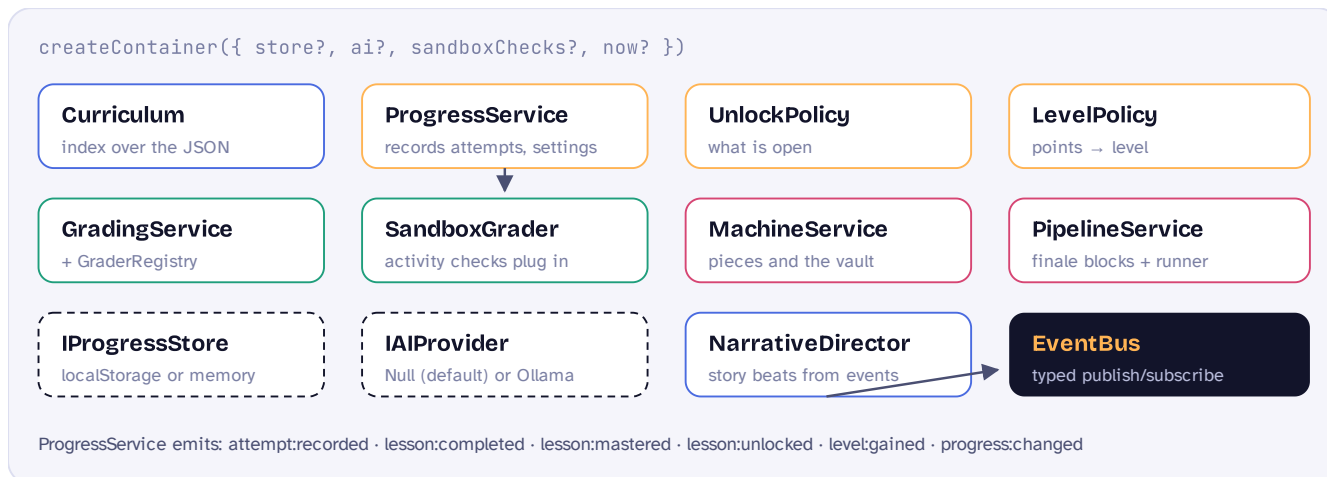


Figure 3. The composition root. Dashed boxes are interfaces with swappable implementations. The UI re-renders on `progress:changed`.

Services and what each owns

SERVICE	RESPONSIBILITY	DOES NOT
Curriculum	Read-only index over the compiled bundle: worlds, lessons, level thresholds, next open lesson.	Know about progress.
ProgressService	Records attempts and settings, saves through the store, imports and exports, emits events.	Decide what is open or what level she is.
UnlockPolicy	Computes each lesson's status from records, pace and hidden worlds.	Store anything.
LevelPolicy	Turns cleared lessons into points and a level.	Store a level; it is always recomputed.
GradingService	Scores answers through one grader per question type; applies the puzzle mode.	Know about screens or storage.
MachineService	Maps each lesson to its jigsaw piece; works out earned, placed, waiting and whether the picture is complete.	Trust stored piece counts.
PipelineService	The finale: which pipeline blocks are built, and running them (live model or recorded demo).	Appear before the picture is complete.
NarrativeDirector	Listens to events and turns them into Iskra's story beats.	Change progress.

What is stored, and where

One JSON object in the browser's localStorage under the key `neuron.progress.v1`. It holds what happened, not what it means: status, level and pieces are recomputed from it on every read.

FIELD	HOLDS
<code>v</code>	Schema version (currently 1). Drives migrations.
<code>records</code>	Per lesson: best score (0–1), attempts, whether the best was without hints, first-completed time, last-seen time.
<code>track</code>	Pace: <code>explorer</code> or <code>builder</code> .
<code>build.placed</code>	Which jigsaw pieces she has placed. The only reward data that is stored, because placing is her own action.
<code>build.theme</code>	The finale project she picked (toys, photos, pets, games).
<code>puzzleMode</code> • <code>hiddenWorlds</code> • <code>pin</code> • <code>motion</code>	Grown-up settings.
<code>learnerName</code> • <code>companionName</code>	Optional display names. Never leave the device.

LOADING

Never crashes on bad data

Old saves are migrated step by step. Unknown or corrupt data falls back to a fresh profile instead of throwing, because a crash on load could not be fixed from the child's side.

IMPORTING

Refuses rather than erases

A backup file that cannot be read or migrated is rejected and nothing changes. Records for lessons this build does not have are dropped.

Offline and security posture

- **No network while learning.** The single file inlines everything; the web app's service worker serves its own files from cache. No analytics, no fonts or images from other sites.
- **Local model is off.** An Ollama adapter exists behind `IAIProvider`, but the container uses `NullProvider`; nothing in this build switches it on.
- **The PIN is a child lock.** It keeps a curious child out of the settings. It is stored in plain text on purpose: there is no uploaded data for it to protect.
- **Cache versioning.** The service worker cache is named `neuron-v2`; changing the name clears old files on the next visit after an update.

What stops a bad build

Each gate catches a class of mistake the others cannot. Several were added after a real bug slipped through, and each note says which.

GATE	CHECKS	COMMAND
Content build	Schema, lesson graph (known ids, world order, no cycles, one start), reading grade ≤ 6.5 for Explorer and ≤ 9 for Builder, no lesson grades an activity it does not have (and a warning if an activity is never graded).	python3 tools/build_content.py
Python tests	64 tests over models, graph, readability, compiler and the real content.	python3 -m pytest tools
Type check	Strict TypeScript across app and generated types.	npx tsc --noEmit
Lint	Code style and the layer rule (domain may not import UI or React).	npm run lint
App tests	222 tests: services, grading, unlocks with hidden worlds, puzzle modes, reward chain, jigsaw meshing, every comic showing its own words, screens.	npm test
Style-class check	Every class used in the source produces CSS. Added after ~60 classes were found silently doing nothing.	part of npm run build
Size gate	Script ≤ 200 KB gzip (now 113.9 KB); CSS ≤ 40 KB (now 9.9 KB).	npm run size
Smoke test	Boots the built single file in jsdom and checks it renders.	npm run smoke
Browser walk	Playwright visits every screen at 390 px and 1280 px: console errors, sideways scroll, tap targets under 32 px, screenshots.	run during review

Build and ship

STEP	COMMAND
Compile content	python3 tools/build_content.py
Web app (for Netlify / PWABuilder)	npm run build → dist/
Single offline file	npm run build:single → dist-single/neuron.html
Everything, CI-style	npm run ci:build && npm test && npm run size && npm run smoke